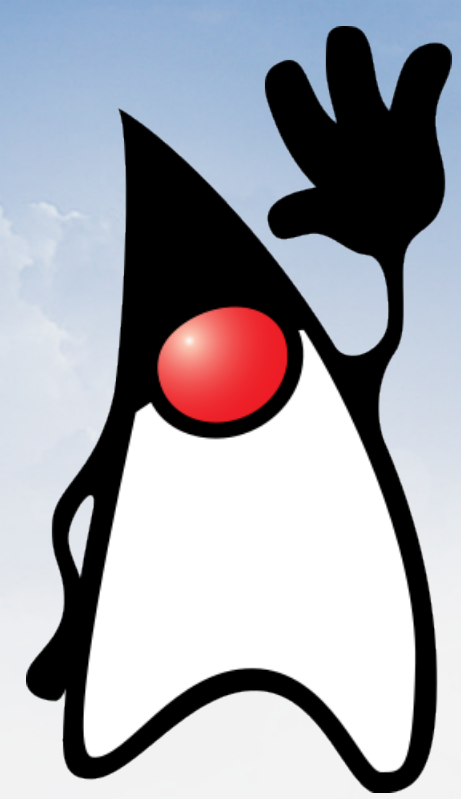




Objects2

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Setting up a Class

Access Modifier →

Main Method →

The name of the class

```
public class Person
{
    //our main method of class Person
    public static void main(String[] args)
    {
        ...
    }
}
```

For now, all the classes are public, same name with the Object name

Minimal working example

```
public class Person
{
    String name;

    //our main method of class Person
    public static void main(String[] args)
    {
        Person p1 = new Person();

        p1.name = "Larry";

        System.out.println(p1.name);
    }
}
```

Confession!

All of our java files were classes (blueprints of objects), even the constructor was predefined. We just didn't use them. We only wrote static methods.

If there is no constructor in the class, Java will implement a default no argument constructor for you. That constructor will not change default values of the instance variables.

Wrong ways to create object

- `Car myCar();`
- `Car myCar;`

This is just a reference. It is supposed to show a memory location for the object.

Object creation should have `new` inside!

Those don't work in Java if we want to create an object!!!

The Class Constructor

- A special method that instantiates an object of the class.
- Every class that we define automatically comes with a default constructor.
 - with no parameters
- It is better you write no argument constructor first!
- Header MUST match the name of the class name
- Can have multiple constructors with different parameters
- Your constructors will be always *public*! What if they aren't?

Car class again

```
package testCars;
public class Car{
    int year; //instance variable
    String brand; //instance variable

    public Car(){
        year=1999;
        brand="Toyota";
    }

    public Car(int givenYear, String gBrand){
        year=givenYear;
        brand=gBrand;
    }

    public static void main(String[] args){
        Car mycar = new Car();
        Car yourCar = new Car(2000,"Volvo");
        System.out.println(mycar.year);
        mycar.year=2005;
        System.out.println(mycar.year);
    }
}
```

What was their initial value? Address? Garbage?

When the **Car()** method is invoked, which is the constructor, the variables are set!

When the **Car(year,Brand)** method is invoked, which is the constructor, the variables are set based on given parameters!

Non initialized variables

- In methods when you don't initialize, Java will give error
 - NEVER EVER use uninitialized variables inside the methods.
 - It will give compilation error
 - What if for `int[] arr`?
- For the instance variables of the objects, there are defaults
 - Object types : `null`
 - Primitives : `0`
 - Boolean: `False`

Car class again

```
package testCars;

public class Car{

    int year; //instance variable
    String brand; //instance variable

    public Car(){
        year=1999;
        brand="Toyota";
    }

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.year);
        mycar.year = 2005;
        System.out.println(mycar.year);
    }
}
```

We can modify any instance variable inside main method or anywhere

We can read any instance variable inside main method or anywhere

Car class again

```
package testCars;

public class Car{

    public int year; //instance variable
    public String brand; //instance variable

    public Car(){
        year=1999;
        brand="Toyota";
    }

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.year);
        mycar.year = 2005;
        System.out.println(mycar.year);
    }
}
```

Still, We can modify any instance variable inside main method or anywhere

Still, We can read any instance variable inside main method or anywhere

Car class again

```
// Car.java
package testCars;

public class Car{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        year = 1999;
        brand = "Toyota";
    }

    public Car(int gyear, String gbrand){
        year=gyear;
        if(year<0){
            year=2000;//default
        }
        brand=gbrand;
    }
}
```

If you write the main inside the Car, you won't have any error!

```
// CarTest.java
package testCars;

public class CarTests{

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.year);
        mycar.year=2005;
        System.out.println(mycar.year);
    } //end of main
}
```

The field Car.year is not visible

As a rule of thumb, generally our tester methods are implemented in a different tester class!

Car class again

```
// Car.java
package testCars;

public class Car{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        year = 1999;
        brand = "Toyota";
    }

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.year);
        mycar.year=2005;
        System.out.println(mycar.year);
    }
}
```

If we write main method inside the Car class, we will be able to access any variable. Even though the instance variables are private.

1999
2005

Access restrictors/modifiers

- public : open to anyone, any class or method in anywhere
 - They can use those methods, use those variables, read or write
- private: only the class itself can reach those variables/methods
- default: Only the classes inside the same package can see it
 - When you don't write anything, it will be default
- protected: Only the classes inside the same package can see it
- In this course, all the instance variables are private
 - If you don't specify any access restrictor, they will be default.
- All methods are public
 - We will see other cases also

This is bad! Another Class can access another Class's instance.

```
public class Person
{
    String name;

    //our main method of class Person
    public static void main(String[] args)
    {
        Person p1 = new Person();

        p1.name = "Larry";

        System.out.println(p1.name);

        Intruder.someMethod(p1);

        System.out.println(p1.name);
    }
}

class Intruder
{
    public static void someMethod(Person p1)
    {
        p1.name = "HAHAHAHAHA";
    }
}
```

The fix. This code won't compile now due to this. We can fix this by creating accessor methods in Person class. We will talk about accessors later.

```
public class Person
{
    private String name;

    //our main method of class Person
    public static void main(String[] args)
    {
        Person p1 = new Person();
        p1.name = "Larry";
        System.out.println(p1.name);
        Intruder.someMethod(p1);
        System.out.println(p1.name);
    }
}

class Intruder
{
    public static void someMethod(Person p1)
    {
        p1.name = "HAHAHAHAHA";
    }
}
```

This won't work now!
Which is good!

Accessor methods

- Known as getters/setters
- Since all the instance variables are private
 - How can we modify them outside of the class?
- Write methods to modify or read them
- Create a bureaucratic level
 - You can stop invalid entries in those methods
 - You can stop accesses to those variables in those methods
- We have now an opportunity to handcuff ourselves
 - Leave ourselves to the mercy of the compiler

Accessor Method
Notice it is non-static

```
public class Person
{
    private String name;

    //our main method of class Person
    public void setName(String name)
    {
        this.name = name;
    }

    public static void main(String[] args)
    {
        Person p1 = new Person();
        p1.name = "Larry";
        System.out.println(p1.name);
        Intruder.someMethod(p1);
        System.out.println(p1.name);
    }
}

class Intruder
{
    public static void someMethod(Person p1)
    {
        p1.setName("HAHAHAHA");
    }
}
```

Car class again

```
// Car.java
package testCars;

public class Car{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        year = 1999;
        brand = "Toyota";
    }

    public int getYear(){
        return year;
    }

    public void setYear(int givenYear){
        Year = givenYear;
    }
}
} //end of class
```

```
// CarTest.java
package testCars;

public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.getYear());
        mycar.setYear(2005);
        System.out.println(mycar.getYear());
    } //end of main
}
```

1999
2005

Still What is the use of it? We have made our job harder?

```
// Car.java
package testCars;
public class Car{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        setYear(1999);
        brand="Toyota";
    }

    public int getYear(){
        if(year<=0){ return -1; }
        return year;
    }

    public void setYear(int givenYear){
        if(givenYear<= 0){
            System.out.println("Year is invalid");
        }else{
            year=givenYear;
        }
    }

}

} //end of class
```

```
// CarTest.java
package testCars;

public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.getYear());
        mycar.setYear(0);
        System.out.println(mycar.getYear());
    } //end of main
}
```

```
1999
Year is invalid
1999
```

```
// Car.java
package testCars;
public class Car{

    private int year; //instance variable
    private String carLockPassword;
    private boolean unlocked;

    public Car(){
        year=1999;
        carLockPassword="letmein";
        unlocked=false;
    }

    private String getPassword(){
        return carLockPassword;
    }

    public void tryToUnlock(String p){
        unlocked = p.equals(getPassword());
    }
} //end of class
```

I could have created a setPassword() which prints out “Once it is set, never changes!!!”

```
// CarTest.java
package testCars;

public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.getPassword());
        mycar.setPassword("0");
        mycar.tryToUnlock("test101");
    } //end of main
}
```

I have done 2 stuff here

- You cannot read password, it is private!
- You cannot change password, there is no such method!

Templates for getters/setters

```
public void setSth(VariableType var){  
    varOfClass=var;  
}
```

```
public VariableType getSth(){  
    return varOfClass;  
}
```

Returning Objects from Methods

```
package carTests;

public class Car{

    private int year=1999;
    // do not do that

    public Car(int year){
        this.year = year;
    }

    public void roar(){
        System.out.println("roar "+year);
    }
}
```

initializing instance variables
outside of the constructors is a bad
practice.

```
package carTests;

public class CarTest{

    public static void main(String[] args){
        int myage = 36 ;
        Car myCar; // null by default
        myCar = specialCarMod(myage);
        myCar.roar();
    }

    public static Car specialCarMod(int age){
        int carAge = 0;
        if( age %2 == 0) { //even age
            carAge = (int) Math.pow(2, age/2);
        }else{ //odd age
            carAge = (int)( Math.sqrt(age) ) + 2000;
        }
        return new Car(carAge);
    } //end of static method

} //end of class
```

Truth about pass by stuff

Time to confess!

The creators have decided on an clever idea
Pass by value which is a copy of reference

Inside methods

- You cannot modify primitive data types!
- You **cannot** null the objects
 - but you can change the objects!

```
package testCars;

public class Car{

    public int year; //instance variable
    public String brand; //instance variable

    public Car(int year,String brand) {
        this.year = year;
        this.brand = brand;
    }
}
```

```
0
VW
BMW
```

You changed the property of the object. Reference is still same.

You changed the reference of String so the change is not visible.
Copy of the reference is passed to methods.

```
package testCars;
public class CarTester{
    public static void main(String[] args){
        Car myCar = new Car(1999,"VW");
        fixYear(myCar);
        System.out.println(myCar.year);
        changeStr(myCar.brand);
        System.out.println(myCar.brand);
        fixBrand(myCar);
        System.out.println(myCar.brand);
    }

    public static void fixYear(Car c){
        c.year=0;
    }

    public static void changeStr(String name){
        name = name+"Corolla";
    }

    public static void fixBrand(Car c){
        c.brand="BMW";
    }
}
```

```
package testCars;

public class Car{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        year = 1999;
        brand = "Toyota";
    }

    public getYear(){
        return year;
    }

}
```

You changed the reference of Car so the change is not visible. Copy of the reference is given to methods.
We still see the year.

Can we “nullify” the reference in the parameter?

```
package testCars;

public class CarTester{
    public static void main(String[] args){
        Car myCar = new Car();
        System.out.println(myCar.getYear());
        nullIt(myCar);
        System.out.println(myCar.getYear());
    }

    public static void nullIt(Car mycar){
        mycar = null;
    }

}
```

1999
1999

Object methods

- toString
- equals
- hashCode
- clone
- etc...

As a rule of thumb always write toString and equals methods inside a new class!

toString

- We have seen this method on countless occasions now.
- String representation of the object
 - *brand year is 1999, Car is not runnable*
- Human readable format of the object
 - Many uses, including debugging, reporting etc..
- Makes our job easy if we want to debug
- Always write toString() method for your class
- Make it always *public String toString()*
- DO NOT MAKE IT VOID OR PARAMETRIZE IT!
 - What happens if you do?

Car class again

```
// Car.java
package testCars;

public class Car{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        year=1999;
        brand="Toyota";
    }

    public String toString(){
        return "Brand is " +brand+
               " Year is " + year;
    }
}
```

when you don't write toString() inside print, java does that for you!

```
// CarTest.java
package testCars;

public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.toString());
        System.out.println(mycar);
    } //end of main
}
```

```
Brand is Toyota Year is 1999
Brand is Toyota Year is 1999
```

```
package carTest;
public class Car {
    private String engineType;
    private int price;

    public Car(){
        price = 99;
        engineType = "gasoline";
    }
    public Car(int givenprice) {
        engineType = "electric";
        price = givenprice;
    }
    public int getPrice() {
        return price;
    }

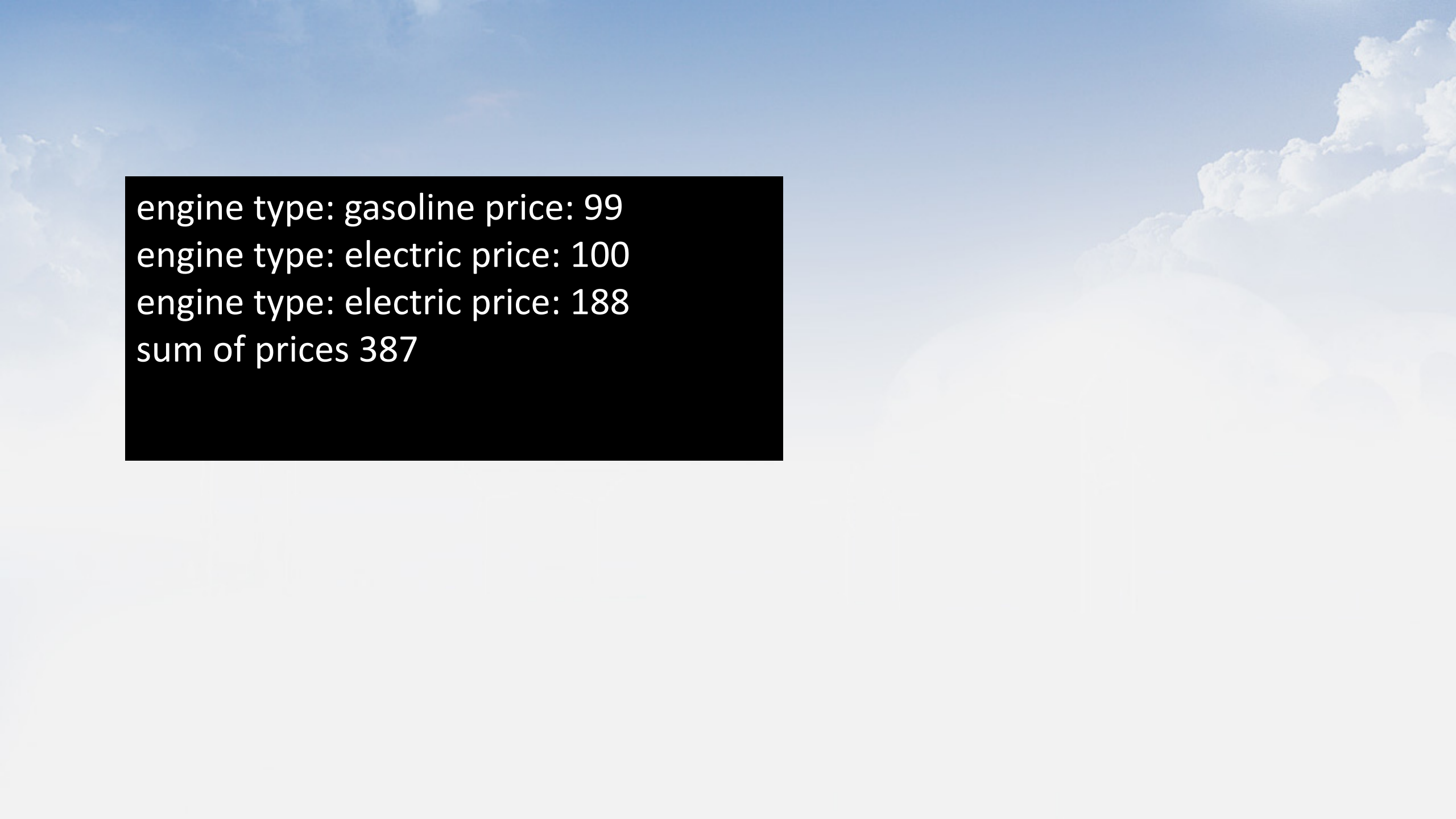
    public String toString() {
        return "engine type: "+engineType+
            " price: "+price;
    } //end of toString
}
```

```
package carTest;
import java.util.ArrayList;

public class CarTest {
    public static void main(String[] args) {
        Car mycar = null;
        mycar = new Car(100);
        Car yourcar = new Car();

        ArrayList<Car> garage = new ArrayList<>();
        garage.add(yourcar);
        garage.add(mycar);
        garage.add(new Car(188));

        int sum=0;
        for(Car c:garage) {
            System.out.println(c);
            sum += c.getPrice();
        }
        System.out.println("sum of prices "+sum);
    } //end of main
}
```



```
engine type: gasoline price: 99  
engine type: electric price: 100  
engine type: electric price: 188  
sum of prices 387
```